Regression Line

used when data is **numeric**.

Linear regression equation:
y = wx + b

- `w` (weight)
- `b` (bias)

Parameters vs Hyperparameters

Parameters: the set of variables that frequently change during the training process (fine-tuning) (weight,bias)

Hyperparameters: the set of constants that do **not** change during training; their values are specified *prior to* the training process.

- Examples: number of epochs, learning rate (η), number of parameters.

The Simple (Perceptron) Trick — Pseudocode

Idea: Consists of slightly rotating and translating the line in the direction of the point, to move it closer.

Pseudocode (training a single point):

Input:

- A point (x, y)
- Line equation: m, b
- Learning rate η = 0.1, 0.01, 0.001

Output: Updated line equation m, b

The Absolute Trick — Algorithm

Compute the predicted value:
$\hat{p} = m \cdot r + b$

If $p > \hat{p}$ (point is above the line):

$m' = m + \eta \cdot r$

$b' = b + \eta$

Else if $p < \hat{p}$ (point is below the line):

$m' = m - \eta \cdot r$

$b' = b - \eta$

Return: $\hat{y} = m'x + b$

Overfitting and Underfitting

How do we know we're selecting the right model?

Steps:

1. Divide the dataset (DS) into **Training** and **Testing** sets (common splits: 70/30 or 80/20)
2. Train the model
3. Find the **train error** (loss function)
4. Test the model
5. Find the **test error**
6. Compare both train error and test error

Example comparisons (○ = point for test):

- High-degree polynomial fit (wiggly curve): Train error = 0, Test error = high → **overfitting**
- Well-balanced curve fit: Train error = low, Test error = low → **good fit**
- Straight line (too simple): Train error = high, Test error = high → **underfitting**
- Overfit example: $y = w_1x_1 + w_2x_2 + \ldots + w_{10}x^{10} + b$
- Underfit example: $y = w_1x + b$

Underfitting : memorizes the shape of data but doesn't know the general pattern → high test error.

Overfitting: simplifying complexity of the equation (model) means making some weights = 0.

Regularization

Purpose: Used to overcome the wrong choice of a (too) complex model → helps choose the best model → validate.

Example of an overly complex model:

$y = w_{10}x^{10} + w_9x^9 + w_8x^8 + ... + b$

10 parameters affecting the model.

To overcome overfitting and reach a well-fit model:

- **Simplify the complex model** (reduce the complexity / reduce the number of parameters)
- This is done **during training** (the model is trained once)

During training, we try to:

- Improve the **performance**, and
- Reduce the **complexity**

performance + complexity = regularization error

Measure performance and complexity:

- Measure performance and complexity using two different error functions
- Add them together to get a more robust error function

L1 & L2 Norms (Regularization Term)

Used to measure **complexity** and **performance**, and to update weights.
Regularization error

L1 norm:

- Sum of absolute weights
- Lasso regression

L2 norm:

- sum of squared weights
- Ridge regression
- Gradient descent

Note: L1/L2 does **not** include the bias term

reg_error = pred_error + L1

reg_error = pred_error + L2

Compare between L1 and L2:** the higher the value → the more complex the model.

Example

M1: $\hat{y} = 2x_3 + 1.4x_7 - 0.5x_8 + 8$

M2: $\hat{y} = 22x_1 - 103x_2 - 14x_3 + 109x_4 - 93x_5 - 203x_6 + 87x_7 - 55x_8 + 378x_9 - 25x_{10} + 8$

For M1:

L1 = |2| + |1.4| + |-0.5| = 3.9

L2 = $(2)^2 + (1.4)^2 + (-0.5)^2 = 6.21$

For M2:

L1 = |22| + |-103| + |-14| + |109| + |-93| + ... + |-25| = 1089

L2 = $(22)^2 + (-103)^2 + (-14)^2 + (109)^2 + (-93)^2 + ... + (-25)^2 = 227131$

M2 has much higher L1/L2 values → M2 is far more complex than M1.